

Computer Science Independent Study

Investigating LLM Generated Manim Videos for Education

Syed Ali Haider

sh6070@nyu.edu

New York University Shanghai

May 15, 2025

1 Introduction

1

Large language models (LLMs) have recently shown promising capabilities in tasks requiring formal reasoning, code generation, and natural language explanation. Within the domain of mathematics and theorem proving, this has led to growing interest in how such models might assist or automate explanation tasks—particularly in few-shot settings where limited examples are available. Except for recently *TheoremExplainAgent*, most existing approaches are still grounded in static prompt engineering and one-shot generation, which often yield brittle or incomplete results. These methods typically fail to reflect the iterative, feedback-driven reasoning process.

In this report, we present an overview of recent work in the emerging intersection of code generation, theorem explanation, and agentic LLM workflows. We analyze systems such as *TheoremExplainAgent*, *Generative Manim*, *TMA.live*, and *3Blue1Brown*, each of which contributes uniquely to the ecosystem of explainable, multimodal, or visually enhanced mathematical reasoning. By examining their system designs, modes of interaction, and pedagogical thinking, we aim to contextualize our own agentic pipeline within this broader landscape.

This work is more of an exploration, and it is twofold. First, we document and synthesize key insights from recent literature and toolkits, identifying common strategies and limitations in multimodal theorem explanation. Second, we present an early-stage agentic workflow that attempts to operationalize few-shot theorem explanation through a modular architecture involving multiple LLM agents. Each agent specializes in a sub-task—code generation, output verification, or iterative refinement—with the goal of improving the quality of final explanations through feedback-aware coordination.

As this work is in progress, we do not claim to offer conclusive empirical benchmarks. Instead, we focus on qualitative evaluation, using human judgment to assess the clarity, correctness, and explanatory usefulness of outputs with and without agentic feedback loops. Our experiments were conducted on various theorems but we have

¹Submitted to **Prof. Hongyi Wen** (Advisor), *New York University Shanghai*

include the well-known Pythagorean’s Theorem—to serve as illustrative minimal proof of concept. Through this, we identify early signals suggesting that iterative feedback can enhance layout quality, even under few-shot constraints.

This report is intended as both a survey of relevant prior work and a documentation of initial findings in our ongoing exploration.

2 Overview of Current Works

The landscape of AI-driven STEM explanation has evolved rapidly, with recent systems leveraging large language models (LLMs) to orchestrate multimodal outputs that combine narrative, code, and animation. In particular, TheoremExplainAgent pioneers an agentic two-stage pipeline for generating full-length theorem explanation videos, while tools like Generative Manim and TMA.live showcase serverless, on-demand text-to-animation services built atop the open-source Manim engine. The foundational Manim library underlies all of these innovations, and the 3Blue1Brown youtube channel where these videos take inspiration from. Collectively, these works illustrate a move toward seamless, reproducible production of high-quality educational content via LLMs and programmatic animation. However, there is a major problem in the layout flaws and overlaps of elements in the generated videos in many of these works.

2.1 LLMs

Recent advances in LLMs and vision-language models (VLMs) have unlocked powerful capabilities for multimodal understanding. Models like GPT-4 [13], Gemini [6], Llama [11], Claude 3.5 [1], and DeepSeek [3] demonstrate strong performance in processing complex text and visual inputs within unified architectures [17]. These breakthroughs have enabled transformative applications in visual understanding [8], code generation [12], and structured concept reasoning.

2.2 LLM in Visualizations

Recent advances in LLM-driven data analysis have enabled systems to generate visualizations from natural language queries by translating them into executable code [10]. Frameworks like Drawing-Pandas [5] established benchmarks for evaluating code-based plotting in tools like Matplotlib and Seaborn, while works such as MatPlotAgent [16] demonstrated how structured prompting improves visualization accuracy. Subsequent efforts like PlotGen [7] introduced iterative refinement through multimodal feedback, enhancing robustness.

2.3 3Blue1Brown: Inspiration Through Programmatic Visualization

The 3Blue1Brown project [14], developed and maintained by Grant Sanderson, has played a foundational role in the intersection of mathematics education and program-

matic animation. As one of the most widely viewed and pedagogically influential mathematics channels on the internet, 3Blue1Brown is distinguished by its commitment to intuitive, narratively guided visual explanations of advanced mathematical concepts. Rather than relying on static diagrams or physical whiteboards, the videos produced under this brand employ continuous, smoothly interpolated vector animations that reveal the logic of mathematical structures over time. This distinctive aesthetic is not merely ornamental but is deeply tied to cognitive engagement, allowing viewers to “see” the dynamics of a proof or process in ways that enhance conceptual retention and procedural fluency.

To support this novel form of mathematical communication, Sanderson created the *Manim* (Mathematical Animation Engine) library [4], a Python-based domain-specific language designed for scripting mathematically rich animations. Manim abstracts away low-level graphics complexity while offering expressive primitives for objects such as equations, number lines, coordinate systems, and geometric transformations. The library’s open-source release catalyzed a vibrant community of contributors and users, leading to the emergence of the Manim Community Edition (MCE) [4], a maintained and production-ready fork that enhanced stability, extensibility, and documentation. The Manim community now includes researchers, educators, and developers who use the engine not only for video production but increasingly in automated and AI-driven workflows.

In this way, 3Blue1Brown functions both as a proof-of-concept and as a design ontology for AI-generated educational content. Its success demonstrates the viability of programmatic animation as a communication modality and sets a de facto standard for visual clarity and narrative structure. The influence of 3Blue1Brown is directly evident in recent agentic systems such as *TheoremExplainAgent* and *TMA.live*, both of which inherit and extend Manim’s rendering infrastructure to support end-to-end theorem explanation pipelines powered by large language models.

2.4 Generative Manim

Generative Manim [2] provides an open-source, web-based text-to-video service that abstracts away both prompt engineering and manual coding. Its pipeline consists of a Prompt Generator, which uses LLMs (including GPT-4o and fine-tuned GPT-3.5 variants) to translate natural-language descriptions into Manim Python scripts, and a Render Engine that executes these scripts in containerized environments and streams the resulting frames back to the user in near real-time. Users select from a registry of model backends—trading off domain specialization (e.g., physics-tuned models) for cost and latency—and receive a single API endpoint that unifies prompt-to-video generation under one call.

The novelty of Generative Manim lies in its modular architecture and community-driven extensibility. By decoupling the prompt-to-code and code-to-video stages, the system supports rapid integration of new LLMs and custom render settings without altering the core pipeline. This design democratizes access to programmatic animation: educators and researchers with no prior coding background can produce complex visualizations through simple prompts, fostering broader adoption of AI-driven educational content creation.

2.5 TMA.live

TMA.live [15] positions itself as an on-demand explainer service, generating narrated Manim animations for arbitrary STEM queries within two minutes of user input. Behind the scenes, grounded LLMs (such as Gemini and Claude) generate Manim code that is rendered on serverless GPU instances provided by Modal.com. The front end, hosted on Fly.io, streams partial video frames via the HLS protocol as rendering completes, offering near-real-time playback. An integrated OpenAI text-to-speech engine synchronizes narration tracks, producing fully narrated video lessons without human intervention.

The system’s primary contribution is its seamless user experience: no account creation or manual pipeline orchestration is needed. By combining grounded language models, serverless rendering, and edge streaming, TMA.live demonstrates a scalable blueprint for real-time educational video generation, highlighting trade-offs between grounding fidelity, rendering parallelism, and end-user latency.

2.6 Manim

Originally developed by Grant Sanderson for the 3Blue1Brown YouTube channel, Manim is a Python-based domain-specific library that enables precise, programmatic animation of mathematical concepts. Users compose *Scenes* by instantiating *Objects* and applying *Animations*, leveraging vector graphics and LaTeX integration. The rendering backend converts these high-level descriptions into frame sequences via Cairo and OpenGL, producing videos or GIFs with configurable resolution and frame rate through a command-line interface. In 2020, the community-driven Manim Community Edition [4] fork emerged to improve stability, testing coverage, and documentation, broadening the library’s user base beyond its originator’s research lab.

Manim’s enduring impact stems from its open-source ethos, permissive MIT license, and extensible plugin ecosystem (e.g., Manim Physics). It has become the de facto engine for programmatic math animations, powering millions of views and influencing subsequent AI-driven animation pipelines.

2.7 TheoremExplainAgent (Ku et al., 2025)

In their paper, Ku et al. introduce TheoremExplainAgent (TEA) [9], an agentic AI system for generating multimodal video explanations of mathematical and scientific theorems. The architecture uses two specialized LLM-based agents. The first, called the planner agent, takes a theorem statement as input and constructs a high-level storyboard by segmenting the explanation into sequential scenes. It then refines each scene by writing the narrative text and specifying the visual elements, animations, and transitions needed for that segment. The second agent, the coding agent, receives these detailed scene descriptions and generates executable Python code using the Manim animation library, simultaneously producing a synchronized voiceover via text-to-speech. After assembling the scripts, the system runs them to render the final video. If a script fails to execute correctly, the coding agent inspects the error and iteratively revises the code up to a set retry limit; if the limit is exceeded, that video generation attempt is marked as unsuccessful.

To improve code quality, TEA incorporates an agentic retrieval-augmented generation (RAG) process. The system treats the Manim documentation as an external knowledge base and issues targeted retrieval queries at multiple stages of the workflow. For instance, during storyboarding the planner agent may retrieve illustrative examples or related concepts, and during coding the agent fetches relevant Manim code snippets or usage patterns; it also queries documentation to diagnose and fix any errors that occur. Retrieved information is cached to avoid duplication, and a relevance filter ensures only the most pertinent results are used. This multi-step RAG strategy helps the agents use valid Manim functions and follow established animation conventions, thereby enhancing the correctness of the generated content.

For evaluation, the authors curate TheoremExplainBench (TEB) , a dataset of 240 theorems drawn from OpenStax and LibreTexts sources. These theorems cover four STEM disciplines (Computer Science, Chemistry, Mathematics, Physics) and are evenly split into three difficulty levels (80 each of Easy, Medium, Hard). To score the AI-generated videos, Ku et al. define five automated metrics corresponding to key dimensions of explanation quality: Accuracy & Depth , Visual Relevance , Logical Flow , Element Layout , and Visual Consistency . The first three dimensions assess the correctness and completeness of the explanation (e.g. whether the narration precisely justifies the theorem), while the last two gauge the perceptual quality of the animation (e.g. whether on-screen elements are clearly arranged without overlap).

In practice, TEA’s outputs are evaluated by extracting the video transcript and representative frames. The textual metrics (Accuracy & Depth, Logical Flow) are computed by scoring the narration: GPT-4o compares the transcript against factual/theoretical criteria. For the visual metrics, key frames are extracted and scored: GPT-4o judges how well each frame’s imagery aligns with the theorem (Visual Relevance) and whether graphics are well-positioned (Element Layout). A vision-language model (Gemini 2.0-Flash) analyzes short video segments to quantify animation smoothness (Visual Consistency). The system combines the five scores by taking their geometric mean to produce an overall quality score. The authors validate this automated scoring by a human study: twelve experienced STEM annotators rated a sample of 40 generated videos (10 per discipline) on the same five dimensions. The human ratings (0, 0.5, or 1 per dimension) showed strong correlation with the automated metrics (Spearman correlation) and high inter-rater agreement (Krippendorff’s alpha), confirming that the metrics reliably capture quality.

In experiments on TheoremExplainBench, each agent (planner and coder) is instantiated with the same language model. The study evaluates OpenAI’s GPT-4o, DeepMind’s Gemini 2.0-Flash, Anthropic’s Claude 3.5 Sonnet v1, and a smaller model called o3-mini. As a baseline, the authors tried a non-agentic approach (one model writing the entire script), but it produced only very short videos (on the order of 20 seconds), underscoring that the multi-agent planning is necessary for long-form explanations. Under the agentic framework, the o3-mini agent achieved the highest performance. For example, o3-mini completed the video for about 93.8% of the theorems (a success rate of 93.8%), whereas GPT-4o succeeded on only 55% of cases, and Gemini and Claude barely produced any complete videos. In terms of the five metrics, videos generated by o3-mini scored on average about 0.77 overall, essentially matching the average quality of human-produced Manim explanation videos. The other agents scored similarly or

slightly lower (and often failed on harder theorems). These results indicate that TEA can generate coherent multi-minute videos for a wide range of topics, though performance does vary (for instance, chemistry theorems tended to be more difficult for the system).

A closer analysis of the outputs reveals key findings. The most common failure mode was code hallucination : the agents sometimes generate invalid Manim code (calling nonexistent functions or using wrong APIs), causing run-time errors. Other frequent errors include incorrect LaTeX or math syntax and generic bugs (missing imports, wrong variable names). Even in successful videos, minor visual glitches were prevalent: for example, text labels or diagrams occasionally overlapped or appeared at inconsistent sizes, especially in more complex scenes. On the positive side, the structured video explanations helped diagnose reasoning flaws. In one case the model animated a graph with an arrow moving the wrong way – this visual mistake immediately made its conceptual error obvious, whereas the text narration alone had not highlighted the flaw. Overall, Ku et al. conclude that the agentic approach is essential for producing detailed, long-form explanations, and that the multimodal output exposes model reasoning errors that text-only tests often miss. They also note that improving the spatial layout and coding reliability of the animations remains an important challenge for future work. Therefore we plan to focus on the spatial layout aspect as demonstrated ahead.

3 System Architecture

Our system implements a multi-agent workflow for theorem explanation generation through iterative refinement (Figure. 1). The architecture combines chain-of-thought reasoning with three feedback loops: 1) Code validation, 2) Visual consistency checking, and 3) Pedagogical effectiveness refinement. Agents are orchestrated through a pipeline that progresses from theorem analysis to validated video output, with model specialization optimized for cost-performance tradeoffs using GPT-4o (OpenAI) and open-source LLMs (Together.ai).

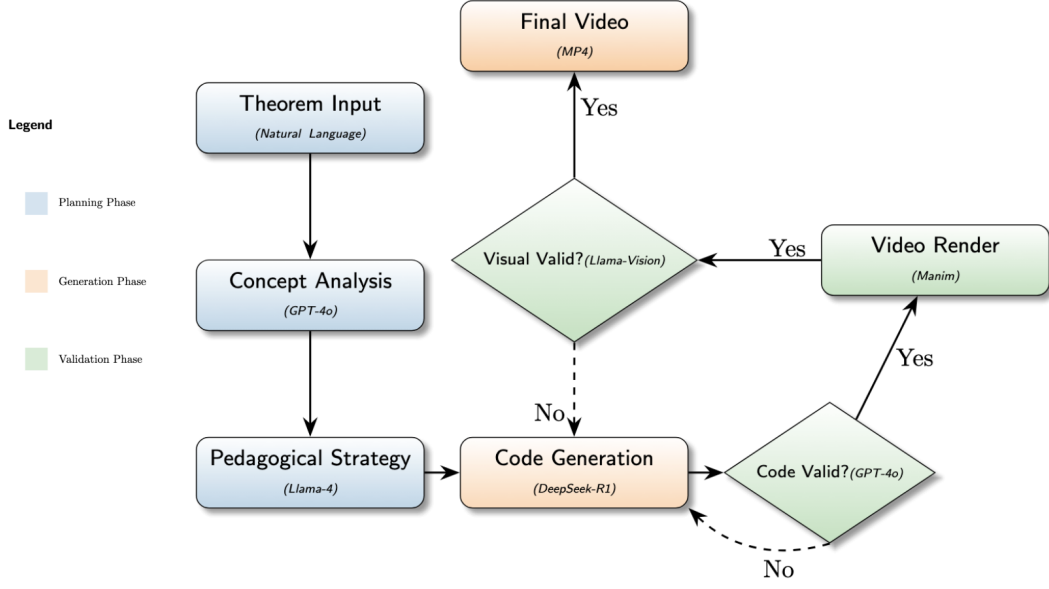


Figure 1: Framework Workflow

3.1 System Design

- **Scoper Agent:** Uses GPT-4o to analyze input theorems through JSON-structured decomposition. Identifies core mathematical components (C_T), visualization requirements (V_T), and common misconceptions (M_T). Output guides subsequent agents' task specialization.
- **Planner Agent:** Implements meta-llama/Llama-4-Maverick for pedagogical strategy design. Generates step-by-step explanation blueprints through chain-of-thought reasoning:
 - Proof decomposition into smaller logical steps
 - Makes study plan
 - Visualization sequencing $\langle V_1, \dots, V_n \rangle$
- **Manim Coder Agent:** Leverages DeepSeek-R1-Distill-Llama-70B for code generation. Transforms planning outputs into executable Manim animations with:
 - MathTex equation scaffolding
 - Scene transition logic
 - Animation timing parameters
 - Few Shot Manim Code

- **Code Checker Agent:** Employs GPT-4o for static analysis of generated code. Validates:

- Mathematical soundness
- Pedagogical alignment with C_T
- Looks for any imports which might cause issues
- Style compliance (layout flaws)
- Stores a binary score 1 or 0 in `flaws` variable

Triggers feedback loop to Manim Coder until `flaws = 0`.

- **Visual Checker Agent:** Uses meta-llama/Llama-Vision for multimodal validation. Processes video frames $\{f_1, \dots, f_k\}$ with:

- Looks for awkward layouts
- Optical flow analysis (Δ_t between scenes)
- OCR validation of equation rendering
- Stores a binary score 1 or 0 in `flaws` variable

Triggers feedback loop to Manim Coder until `flaws = 0`.

Feedback Loops: Two-tiered validation ensures $output_{final} = f_n(\dots f_2(f_1(output_{initial}))$ where f_i represent refinement iterations. The code loop handles syntactic-semantic validation while the visual loop ensures multimodal coherence.

3.2 Prompt Engineering Strategies

Our prompt design follows a hybrid approach combining zero-shot instructions for structured tasks and few-shot exemplars for code generation. We optimize for 1) Task specialization through constrained output formats, and 2) Cognitive scaffolding via progressive disclosure:

- **Scoper Agent (Zero-Shot):** Structured JSON template enforcement through:

Scoper Agent Prompt

Task: Analyze mathematical theorems for core components and visualization needs

Input: $\{theorem\ statement\}$

Requirements:

1. Identify equations/variables
2. List visualization requirements
3. Note common misconceptions

Response Format:

```
{  
  "components": [],  
  "visualization_needs": [],  
  "misconceptions": []  
}
```

Limits free-form responses while permitting theorem-specific analysis.

- **Planner Agent (Chain-of-Thought Zero-Shot):** CoT reasoning triggered by:

Planner Agent Prompt

Task: Develop pedagogical explanation plan

Input: $\{theorem\ components\}$

Requirements:

1. Use CoT reasoning
2. Break into atomic steps
3. Design visualization sequence

Response Format:

- First I will [describe proof foundation]
- Then demonstrate [key transformation]
- Finally connect [theory to applications]

Guarantees stepwise decomposition without example overfitting.

- **Manim Coder (Few-Shot & Role Play):** Code examples, and role play:

Manim Coder Agent Prompt

Task: Generate Manim code with visual annotations

Role: Manim Community v0.19.0 Expert

Input: *{theorem plan}*

Requirements:

1. Include MathTex equations
2. Add `//@manim` directives
3. Follow PEP8 style
4. Avoid make layout overlaps

Example Output:

```
%%manim -qm -v WARNING TheoremExplanation
class TheoremExplanation(Scene):
    def construct(self):
        # @manim: Create equation animation
        equation = MathTex(r"e^{i\pi}+1=0")
        self.play(Write(equation))
```

Maintains syntax correctness while allowing theorem-specific adaptations.

- **Code Checker (Validation Zero-Shot):** Constraint-based prompting:

Code Checker Agent Prompt

Task: Validate generated Manim code

Input: *{generated code}*

Analysis Dimensions:

1. Mathematical accuracy
2. Animation timing (0.5-2s per transition)
3. Scene composition

Response Format:

```
{
  "flaws": [{"type": "timing", "description": "..."},
  "flaw_score": 0 or 1
}
```

Focuses verification on pre-defined quality dimensions.

- **Visual Checker (Few-Shot Frame Analysis):** Multimodal instructions:

Visual Checker Agent Prompt

Task: Validate video frames against theorem explanation

Input: $\{video\ frames\} + \{theorem\ statement\}$

Analysis Criteria:

1. Equation/text legibility
2. Frame-to-concept alignment
3. Transition smoothness
4. Layout Overlaps

Example Failure Case:

- **Frame 12:** Euler's formula equation unreadable
- **Frame 24:** Arrow animation obscures main proof

Response Format:

```
{
  "passed": false,
  "issues": [
    {"frame": 12, "type": "legibility", "
      description": "..."}
  ]
}
```

Feedback Loop Prompts: Iterative refinement uses error-specific priming:

Feedback Loop Prompt

Task: Revise flawed Manim code

Context: Previous attempt failed because: *{flaws}*

Input: *{original code}*

Requirements:

1. Address specific *{specific_issues}*
2. Maintain original theorem intent
3. Add version comments

Response Format:

```
# Revision v2: Fixed timing issues
class RevisedExplanation(Scene):
    def construct(self):
        self.wait(0.5) # Adjusted timing
```

We aim to maintain problem context while preventing hallucination, so this workflow keeps the LLM focused on the task at hand.

3.3 Secure Execution Environment

We employ a sandboxed execution environment to safely execute LLM-generated code while preventing unauthorized system access. The environment restricts built-in functions to a whitelist of safe operations (e.g., mathematical calculations) and blocks dangerous modules like `os`, `sys`, and `subprocess`. Code executes in a restricted namespace that only exposes necessary variables and validated libraries.

4 Case Analysis

We conduct a longitudinal case study of our system’s iterative refinement capabilities using the *Pythagorean Theorem* ($a^2 + b^2 = c^2$) as a constant. Through three development iterations—initial generation (no feedback), code-corrected visually-validated (2nd iteration), the third 3rd iteration. we analyze error propagation and correction across three critical failure modes:

- **Text Legibility:** Font sizing and equation rendering defects
- **Mobject Placement:** Suboptimal spatial organization of visual elements
- **Scene Composition:** Object overlap and misalignment

Using frame-level analysis of identical temporal points across iterations (Figure 2-10), we demonstrate how our feedback loops: 1) Resolve concrete rendering (LaTeX) flaws through targeted code revisions, and 2) Progressively enhance pedagogical effectiveness.

Iteration 1: Baseline Generation

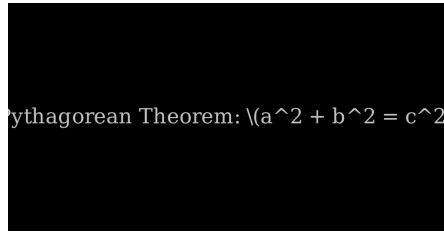


Figure 2: Initial output exhibiting Text Legibility issues

- **Analysis:**

- × *Text Rendering*: Incorrect LaTeX interpolation in MathTex
- × *Text Positioning*: Incorrect positioning of text, it exceeds dimensions

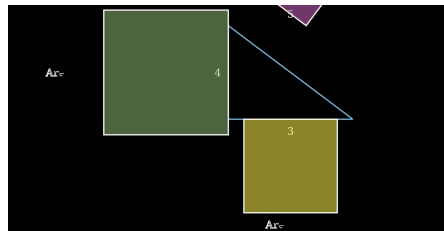


Figure 3: Initial output exhibiting Mobject Placement issues

- **Analysis:**

- × *Object Size*: The squares are not aligned to the edges of the triangle
- × *Object Positioning*: Incorrect positioning of purple square, it exceeds dimensions

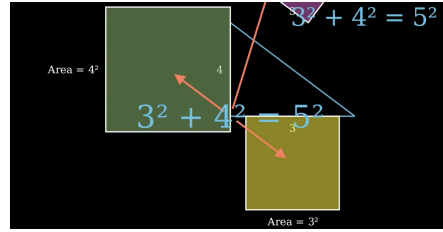


Figure 4: Initial output exhibiting Mobject Placement and Scene Composition issues

- **Analysis:**

- × *Object Positioning*: Incorrect positioning of arrows and it also exceeds dimensions
- × *Text Positioning*: The blue equations are overlapping the background objects,
- × *Logic*: Overall looks chaotic and incomprehensible.

Iteration 2: First Feedback Generation

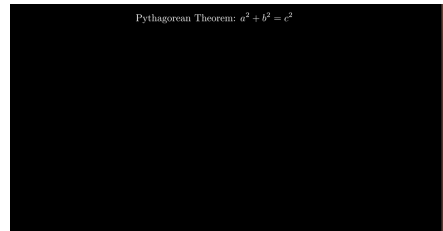


Figure 5: Output exhibiting Text Size Legibility issues

- **Analysis:**

- × *Text Rendering*: Now corrected LaTeX interpolation in MathTex
- × *Text Size*: The size is too small now

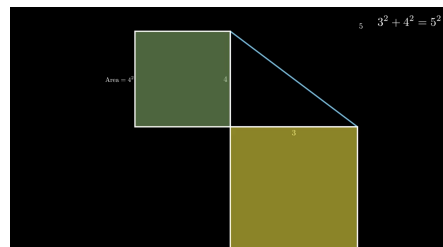


Figure 6: Output exhibiting Mobject Placement issues

- **Analysis:**

- × *Object Size*: Two squares are now perfectly aligned to the edges of the triangle
- × *Object Positioning*: Incorrect positioning of purple square, it now not visible at all

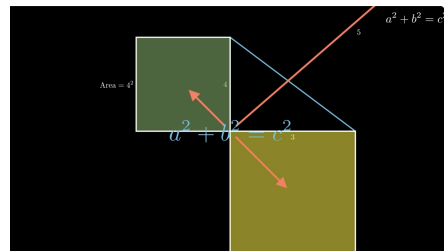


Figure 7: Output exhibiting Mobject Placement and Scene Composition issues

- **Analysis:**

- × *Object Positioning*: Still incorrect positioning of arrows and it also exceeds dimensions
- × *Text Positioning*: The blue equations are overlapping the background objects, but the equation on the right top is now better as it is smaller and not overlapping
- × *Logic*: Overall looks better but still incomprehensible.

Iteration 3: Second Feedback Generation

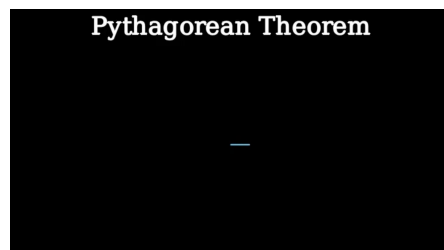


Figure 8: Output with improved Text Legibility

- **Analysis:**

- × *Text Rendering*: Removed LaTeX math completely
- × *Text Size*: The size is perfect now

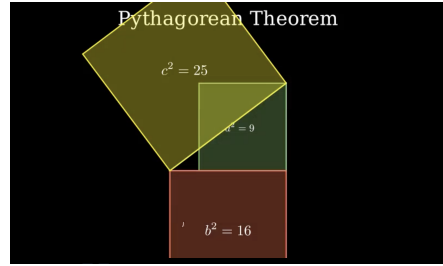


Figure 9: Output exhibiting improved Mobject Placement

- **Analysis:**

- × *Object Size*: Three squares are now perfectly aligned to the edges of the triangle
- × *Object Positioning*: Correct positioning of all square, making it visually apparent the areas of them.
- × *Text Positioning*: Added new area values in the middle of the squares which make it more comprehensible.

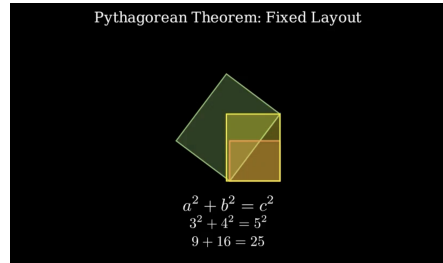


Figure 10: Output with improved Mobject Placement and Scene Composition

- **Analysis:**

- × *Object Positioning*: No arrows now, attempts to move the squares next to each other instead.
- × *Text Positioning*: The equations are clearly legibility with correct $\text{\LaTeX}$ format
- × *Logic*: Overall looks much better and comprehensible.
- × *Text Logic*: In the title now it says "Fixed Layout" which is not desirable

5 Discussion

Our case analysis demonstrates both the potential and limitations of iterative refinement for theorem visualization. While the case analysis shows that three-stage feedback

pipeline successfully addressed critical rendering issues (text legibility, mobject placement, and scene composition), several systemic challenges emerge:

5.1 Key Findings

- **Nonlinear Improvement:** Correction of one defect class (e.g., text rendering) often introduced new issues (e.g., element disappearance), revealing complex trade-offs in constraint satisfaction
- **Semantic Drift:** The final iteration’s removal of mathematical notation (Figure 6) prioritized visual clarity over formal rigor, highlighting tensions between pedagogical goals
- **Human Evaluation:** Human intervention is implicitly required to judge "comprehensibility"

This work demonstrates a proof-of-concept pipeline for iterative refinement of theorem visualizations, we lack the large-scale testing needed to claim generalizability, so we emphasize these are preliminary results requiring further validation. Moving forward, this prototype suggests three key requirements for robust systems: (1) domain-aware validation metrics that preserve mathematical correctness, (2) scalable evaluation across diverse theorem types, and (3) human-in-the-loop quality control for pedagogical alignment.

6 Acknowledgement

I express my gratitude to Professor Hongyi Wen for sponsoring API calls from closed-source models for his insightful discussions and encouragement.

References

- [1] ANTHROPIC. The claude 3 model family: Opus, sonnet, haiku. Online, February 2024. Accessed: 2025-02-11.
- [2] ARIAS, M., AND PUICHING-MEMORY, Generative manim. Web application, 2024.
- [3] DEEPSEEK-AI, LIU, A., FENG, B., XUE, B., WANG, B., AND WU..., B. Deepseek-v3 technical report, 2024.
- [4] DEVELOPERS, M. C. Manim community edition. Documentation website, 2020.
- [5] GALIMZYANOV, T., TITOV, S., GOLUBEV, Y., AND BOGOMOLOV, E. Drawing pandas: A benchmark for llms in generating plotting code, 2024.
- [6] GEMINI-TEAM, REID, M., SAVINOV, N., TEPLYASHIN, D., AND DMITRY... Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context, 2024.
- [7] GOSWAMI, K., MATHUR, P., ROSSI, R., AND DERNONCOURT, F. Plotgen: Multi-agent llm-based scientific data visualization via multimodal feedback, 2025.
- [8] HU, Y., LIU, B., KASAI, J., WANG, Y., OSTENDORF, M., KRISHNA, R., AND SMITH, N. A. Tifa: Accurate and interpretable text-to-image faithfulness evaluation with question answering. *arXiv preprint arXiv:2303.11897* (2023).
- [9] KU, M., CHONG, T., LEUNG, J., SHAH, K., YU, A., AND CHEN, W. Theorem-explainagent: Towards multimodal explanations for llm theorem understanding. *arXiv preprint arXiv:2502.19400* (Feb. 2025).
- [10] LI, G., WANG, X., AODENG, G., ZHENG, S., ZHANG, Y., OU, C., WANG, S., AND LIU, C. H. Visualization generation with large language models: An evaluation, 2024.
- [11] META AI. Llama 4: The beginning of a new era of natively multimodal intelligence. <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>, Apr. 2025. Accessed: 2025-05-20.
- [12] NIJKAMP, E., PANG, B., HAYASHI, H., TU, L., WANG, H., ZHOU, Y., SAVARESE, S., AND XIONG, C. Codegen: An open large language model for code with multi-turn program synthesis, 2023.
- [13] OPENAI. Gpt-4 technical report, 2023.
- [14] SANDERSON, G. 3blue1brown. Website, 2025.
- [15] SHAH, Z. Tma.live. Web application, 2024.

- [16] YANG, Z., ZHOU, Z., WANG, S., CONG, X., HAN, X., YAN, Y., LIU, Z., TAN, Z., LIU, P., YU, D., LIU, Z., SHI, X., AND SUN, M. Matplotlibagent: Method and evaluation for llm-based agentic scientific data visualization, 2024.
- [17] ZHANG, Z., ZHANG, A., LI, M., ZHAO, H., KARYPIS, G., AND SMOLA, A. Multimodal chain-of-thought reasoning in language models. *arXiv preprint arXiv:2302.00923* (2023).